Web Based ML Model Evaluator

Team: 13

Members:

1. SHREYA AJITH (PES2UG24CS480)
2. SANYAM DEVEN HIRAN (PES2UG24CS446)
3. SHREYA PATIL (PES2UG24CS484)
4. SATVIK PHADKE (PES2UG24CS450)

Functional Requirements

Data Ingestion and Preprocessing:

- **FR-1 Dataset Upload:** User should be able to upload their dataset file.
- **FR-2 Dataset Preview:** System should be able to analyze the uploaded CSV and present a dataset summary showing total rows, column names, data types, missing value counts, and a 5 row data preview.
- **FR-3 Preprocessing:** System should be able to preprocess and clean the dataset such as handling missing values.

ML Task Routing & Configuration Module:

- **FR-4 Task Selection:** System should be able to allow the user to select an ML problem type (Classification, Regression, etc) before configuring models.
- **FR-5 Feature & Target Selection (Supervised):** For Classification and Regression tasks, the system should allow the user to select one column as the target variable and one or more columns as features.
- **FR-6 Feature Selection & Hyperparameter Input (Clustering):** For clustering, the system should allow the user to input the number of clusters.
- **FR-7 Train/Test Partitioning:** For Supervised tasks, the system shall provide an option to allow users to adjust the train/test split ratio.

Execution & Evaluation Module

- **FR-8 Task-Specific Algorithm Execution:** System shall execute model fitting based on user selection:
    - Classification: Logistic Regression, Decision Tree Classifier, Random Forest Classifier, K-Nearest Neighbors (KNN).

    - Regression: Linear Regression, Decision Tree Regressor, Random Forest Regressor.
    - Clustering: K-Means Clustering.
- **FR-9 Classification Performance Metrics:** System should compute and display Accuracy, Precision, Recall, F1-Score, and a confusion matrix if needed.
- **FR-10 Regression Performance Metrics:** System should compute and display mean absolute error, Mean Squared Error (MSE), Root Mean Squared Error and R2 score.
- **FR-11 Clustering Performance Metrics & Visuals:** System should compute Silhouette Score and Davies-Bouldin Index, and display a 2D Scatter Plot generated via Principal Component Analysis (PCA) showing cluster assignments.
- **FR-12 Model Comparison Dashboard:** System should allow users to compare evaluation metrics of two or more trained models side by side.
- **FR-2 Robust Error Handling & Data Validation:** The backend should intercept invalid execution states and return structured, human-readable error messages to the UI.

Non-Functional Requirements

- **NFR-1 Pipeline Response Latency:** Ingestion, feature scaling, model fitting, and metric generation for datasets under 10 MB shall execute within 30 seconds per run.
- **NFR-2 Ease of Use:** The web interface should guide the user through a clear process [upload -> select columns -> train model -> display results]
- **NFR-3 Browser Compatibility:** The web application should run smoothly on modern web browsers (Google Chrome, Firefox, Microsoft Edge) without requiring any extra browser plugins.
- **NFR-4 File Limit:** The system should restrict file uploads to a maximum size to prevent server slowdowns.

Jackfruit Phase-1

Software Requirements Specification

for

Web Based ML Model Evaluator

Version 1.0 approved

Prepared by Group 13

PESU

date created: 2/9/2026

Table of Contents

Table of Contents ii

Revision History ii

1. Introduction 1

1.1 Purpose 1

1.2 Intended Audience and Reading Suggestions 1

1.3 Product Scope 1

1.4 References 1

2. Overall Description
3. External Interface Requirements

4. Analysis Models
5. System Features 4
5.1 System Feature 1 4
5.2 System Feature 2 (and so on) 4
6. Other Nonfunctional Requirements 4
6.1 Performance Requirements 4
6.2 Safety Requirements 5
6.3 Security Requirements 5
6.4 Software Quality Attributes 5
6.5 Business Rules 5
7. Other Requirements 5
Appendix A: Glossary 5
Appendix B: Field Layouts 5
Appendix C: Requirement Traceability matrix 6

Revision History

Name	Date	Reason For Changes	Version

Introduction

Purpose

This document's purpose is to demonstrate detailed specifications of the Web Based ML Model Evaluator. Please find the defined functional capabilities, external interface behaviors, system architecture constraints and non functional requirements for the platform. This will serve as a reference to develop, test and verify our project throughout the process.

Intended Audience

This document is intended for the following readers:

- Developers: To implement the backend and UI of the project.
- Testers: To understand the work flow, required inputs, expected outputs and build test cases from the details.
- Evaluator: To review our work.
- Users: The ones who will use the software.

Product Scope

This software is designed to help users to figure out how different types of machine learning models and algorithms work for a selected problem using metrics that help evaluate the most model/algorithm and come up to a conclusion.

Below are some key system capabilities:

- Data ingestion: allows user to upload the dataset that they want to work on.
- Data pre-processing: clean the data before training and train/test split of data.
- Task routing: The software should support various different types of machine learning problem types like classification, regression and clustering.
- Model execution: applying core algorithms like logistic regression, decision trees, KNN, linear regression, etc.
- Performance evaluation: Generate performance metrics such as accuracy, F1 Score, etc which will help the user determine and select the best model and algorithm.
- Visualization: Generate visual outputs like confusion matrix, 2D PCA scatter plots, etc for better readability and understanding.
- Side by side metric comparison enabling users to compare performance between multiple trained models simultaneously.

References

- IEEE Recommended Practice for Software Requirements Specifications.
- Documents created by PESU and slides.

Overall Description

2.1 Product Perspective

The Web Based ML Model Evaluator is a self-contained web application designed to streamline the machine learning workflow. It provides an end-to-end pipeline from data ingestion to model evaluation within a unified browser interface.

2.2 Product Functions

- Dataset uploading and CSV preview generation.
- Data preprocessing, including missing value handling.
- ML task routing for Classification, Regression, and Clustering.
- Feature/target selection and train/test partition adjustments.
- Task-specific algorithm execution across multiple standard models.
- Performance metric computation and visual model comparison.

2.3 User Classes and Characteristics

The primary users are individuals seeking an intuitive, guided process to evaluate machine learning models. Users require a clear workflow that progresses sequentially from dataset upload to result visualization.

2.4 Operating Environment

The application operates entirely within modern web browsers. It must run smoothly on Google Chrome, Firefox, and Microsoft Edge. The system functions independently without requiring any additional browser plugins.

2.5 Design and Implementation Constraints

The system enforces a maximum file upload limit to prevent server performance degradation. Processing times for ingestion, feature scaling, model fitting, and metric generation are strictly constrained to a maximum of 30 seconds for datasets under 10 MB.

2.6 Assumptions and Dependencies

The system assumes that user data is provided specifically in a CSV file format. It depends on a backend architecture capable of intercepting invalid execution states to maintain stability.

External Interface Requirements

3.1 User Interfaces

The user interface provides a step-by-step guided experience: upload dataset, select columns, train model, and display results. The UI displays a detailed dataset summary view showing total rows, column names, data types, missing value counts, and a 5-row preview. It features interactive configuration modules for task selection, cluster inputs, and splitting ratios. Evaluation screens include a model comparison dashboard, performance metrics, and visuals such as 2D Scatter Plots generated via PCA. The UI is also responsible for presenting human-readable error messages.

3.2 Software Interfaces

The system requires a robust backend software interface for data validation and error handling. This backend must capture invalid execution states and transmit structured error data back to the user interface. The software relies on CSV parsing modules to analyze uploaded datasets.

3.3 Communication Interfaces

The application relies on standard browser communication protocols to function seamlessly across Chrome, Firefox, and Edge. Client-server communications must be optimized to ensure that the entire pipeline response latency remains under the 30-second threshold for permitted file sizes.

Analysis Models

Use case diagram:

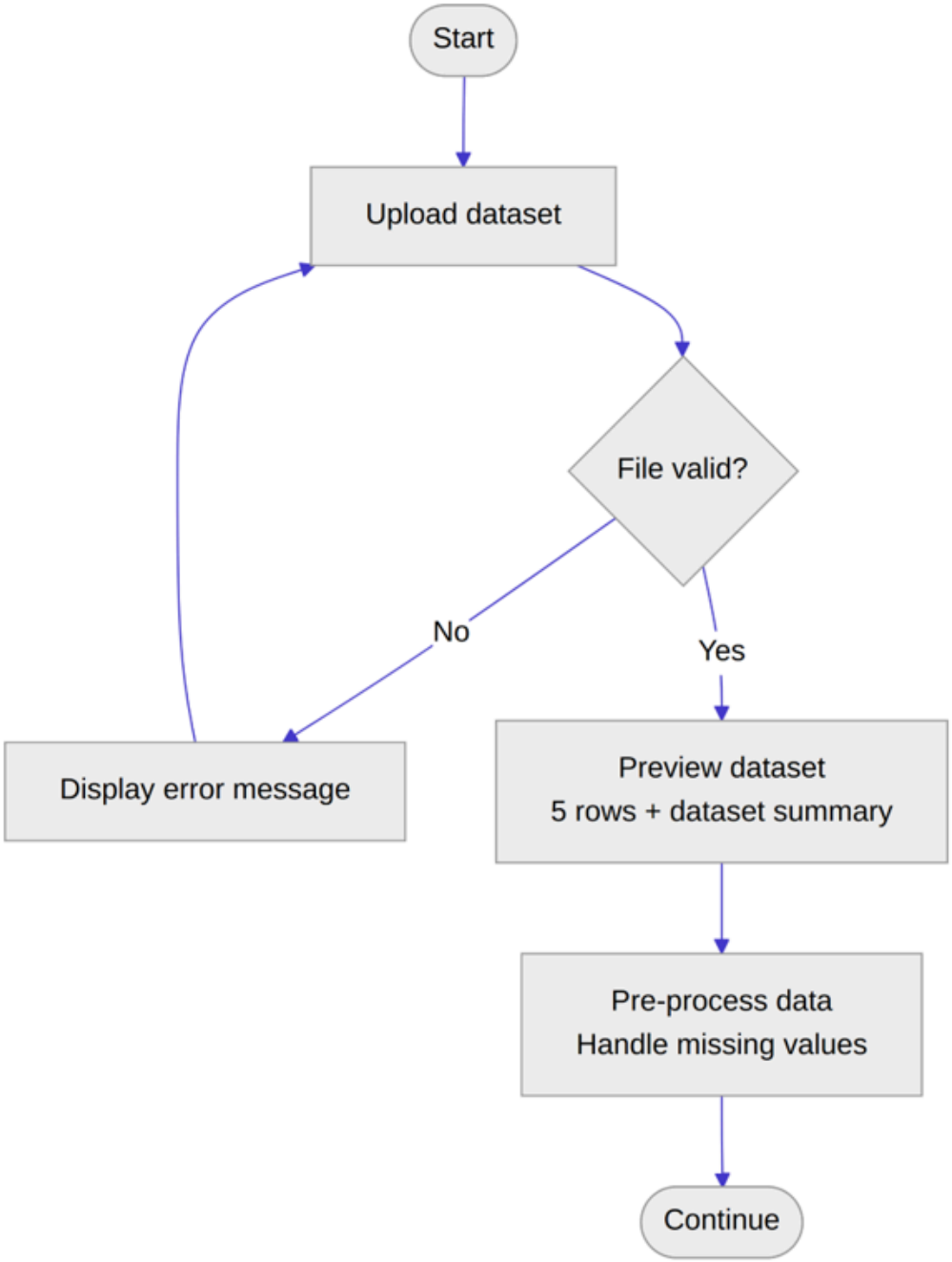


Analysis Model 1: Dataset Ingestion and Preprocessing

Covers:

- Upload dataset
- 5-record preview
- Pre-process data

Activity diagram:

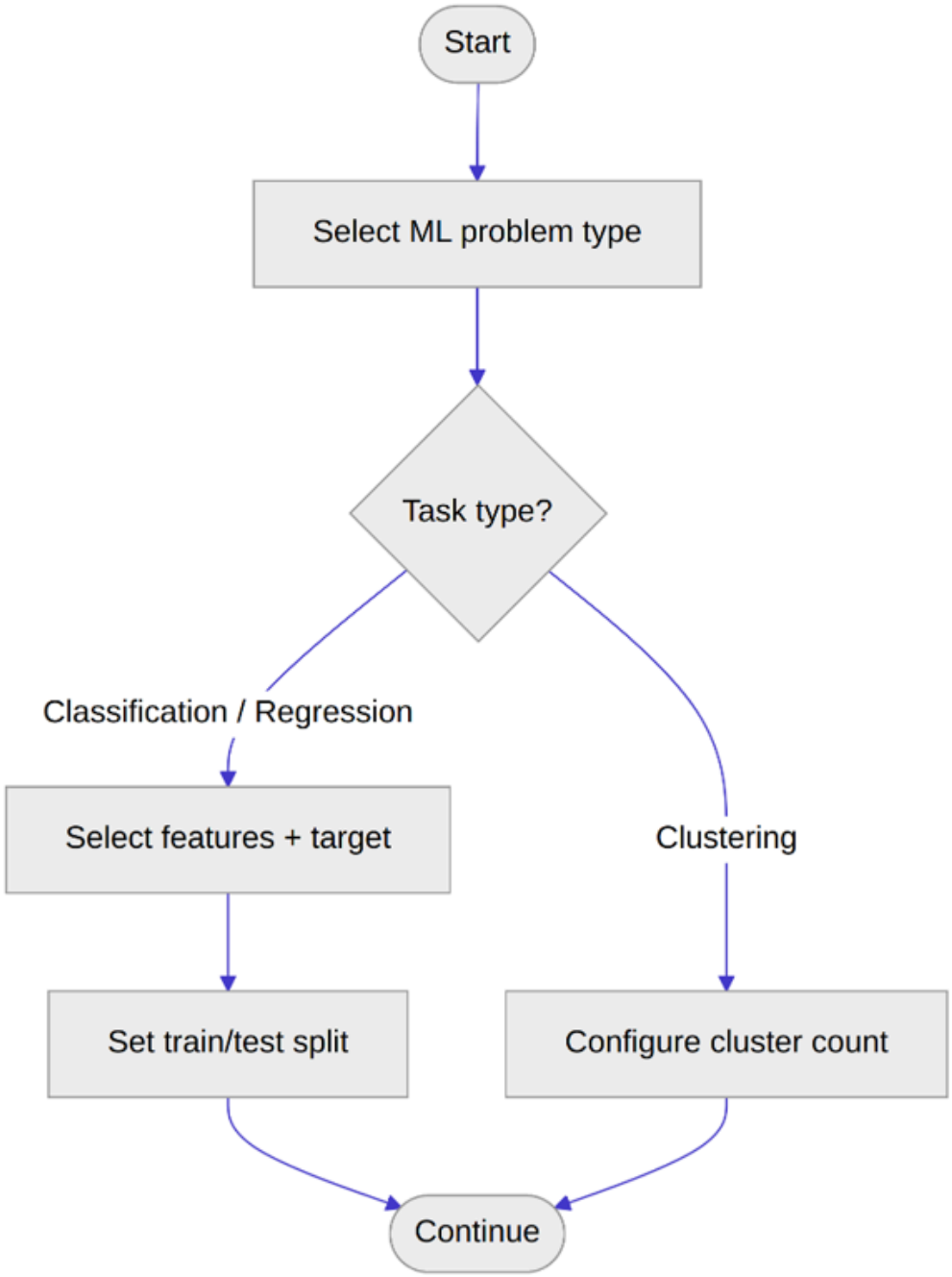


Analysis Model 2: ML Task Configuration

Covers:

- Select ML problem type
- Select features + target (Classification / Regression)
- Configure clustering parameters (Clustering)
- Set train-test split

Activity Diagram:

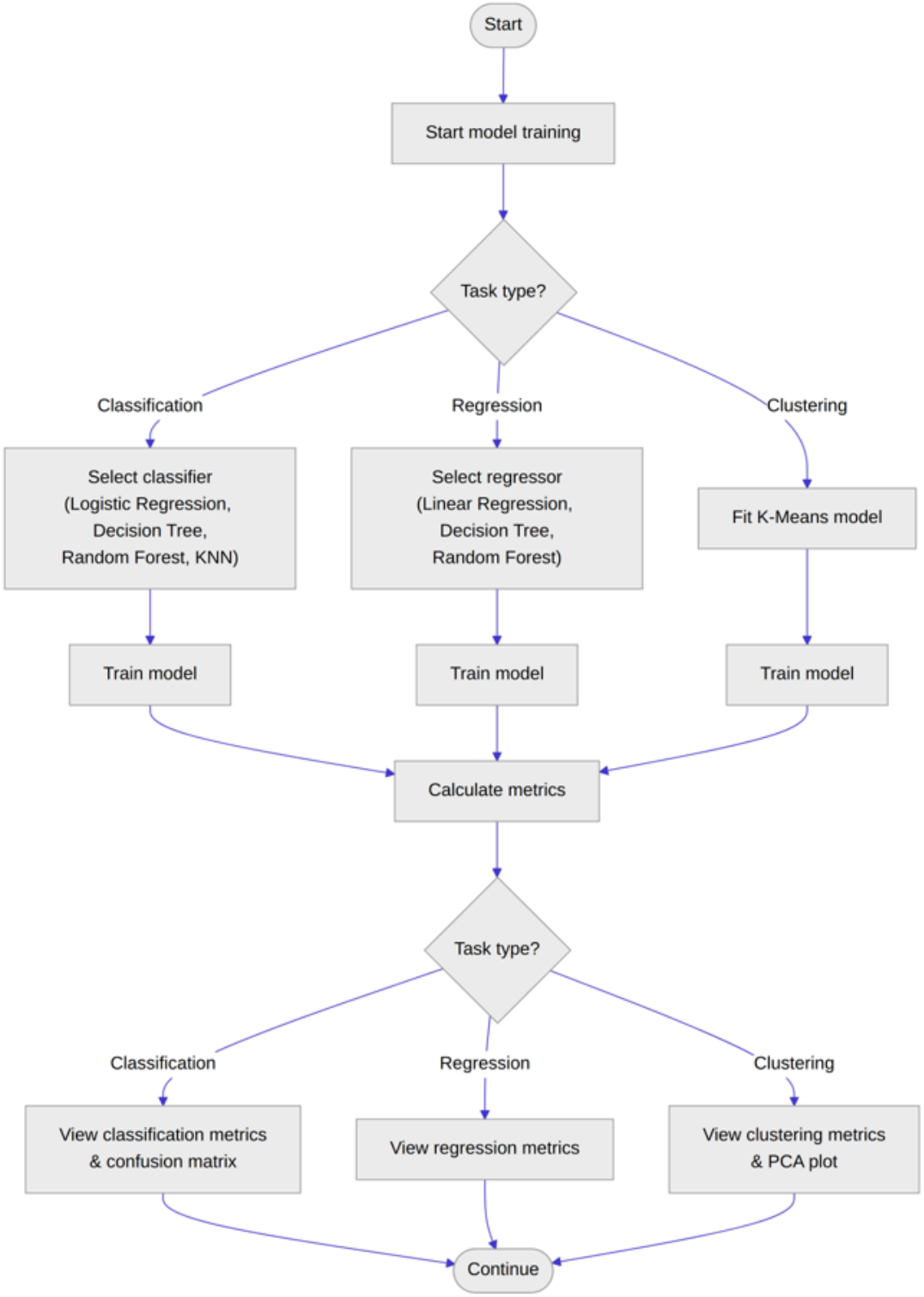


Analysis Model 3: Model Training & Evaluation

Covers:

- Start model training
- Select algorithm (Classifier / Regressor / K-Means) and train
- Calculate metrics
- Classification / Regression / Clustering metric views
- Confusion matrix
- PCA plot

Activity Diagram:



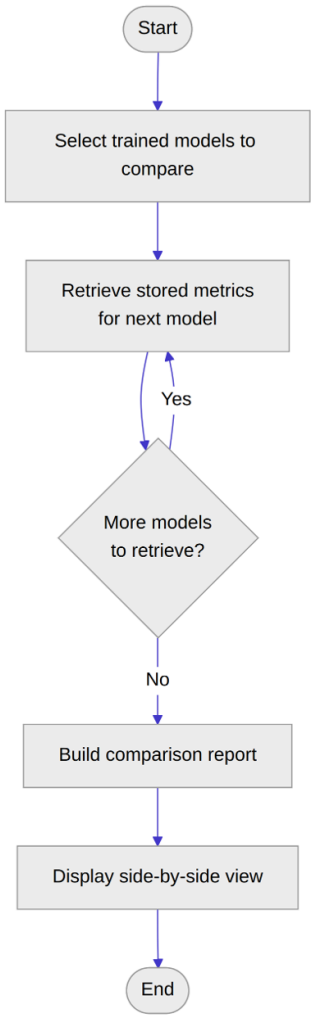
Analysis Model 4: Model Comparison

Covers:

- Trained Models
- Collect evaluation metrics
- Compare models
- Display side-by-side

Activity Diagram

Shows the end-to-end flow: selecting models, looping through each to retrieve its stored metrics, then building and displaying the comparison.



System Features

System Feature 1: Data Ingestion and Preprocessing (5.1)

5.1.1 Description and Priority

Allows users to upload dataset in csv format, preview 5 records from dataset, and handle basic missing values and data pre processing.

Priority: High

Risk involved in uploading dataset is comparatively low whereas risk involved in performing pre processing steps on the uploaded dataset can be challenging since all the datasets have variation and hence the risk involved in this is high. We could involve having certain different types of options for handling missing values such as mean/median/drop rows etc.

5.1.2 Stimulus/Response Sequences

- **User action:** Selects file and clicks upload.
- **System response:** Checks file size, file type, reads the file and previews first 5 records from the uploaded dataset, also performs pre processing and then displays results.

5.1.3 Functional Requirements

REQ-1: The system should allow users to upload CSV files upto a maximum size of 10 MB
REQ-2: The system should display dataset summary of first five records.
REQ-3: The system should provide pre processing options to clean the dataset by handling missing values.

System Feature 2: ML Task Routing and Configuration (5.2)

5.2.1 Description and Priority

Allows users to specify the ML problem type, assign feature and target columns, configure other algorithm parameters, split train/test data.

Priority: High

ML model performance will depend on these inputs from user but implementing this involves low risk.

5.2.2 Stimulus/Response Sequences

- **User Action:** Chooses an ML problem type like classification/regression/clustering.

- **System Response:** Renders relevant column selection drop downs and hyperparameter inputs based on the chosen task.
- **User Action:** Selects target/feature variables and adjusts the train/test split ratio (or specifies the number of clusters for unsupervised tasks).
- **System Response:** Validates selections to prevent invalid states (e.g., selecting the same column as both feature and target) and stores configuration parameters.

5.2.3 Functional Requirements

REQ-4: The system should allow users to select an ML task type like classification, regression or clustering before configuring model parameters.

REQ-5: For supervised tasks like classification and regression, the system should allow users to select one column as the target variable and one or more columns as feature variables.

REQ-6: For clustering tasks, the system should allow users to select feature columns and specify the target number of clusters.

REQ-7: For supervised tasks, the system should allow user to input the train/test data partition ratio.

System Feature 3: Algorithm Execution and Metric Computation (5.3)

5.3.1 Description and Priority

Executes selected machine learning algorithms against the prepared dataset, calculates performance metrics, and generates visualization charts.

Priority: High

This section is very crucial to the software and since, risk involved can be high.

5.3.2 Stimulus/Response Sequences

- **User Action:** Selects target algorithms and clicks execute.
- **System Response:** Fits the selected models, evaluates performance on test/clustered data for files under 10 MB and generates evaluation metrics and charts.

5.3.3 Functional Requirements

REQ-8: The system should execute selected machine learning models based on user choice- logistic regression, decision tree, random forest, KNN for classification problems whereas for regression problems, linear regression, decision tree regressor and random forest regressor and K means for Clustering.

REQ-9: For classification models, the system should calculate and display Accuracy, Precision, Recall, F1-Score and a Confusion Matrix.

REQ-10: For regression models, the system should calculate and display mean absolute error, mean squared error (MSE), root mean squared error.

REQ-11: For clustering models, the system should calculate silhouette score and Davies-Bouldin Index, and generate a 2D PCA scatter plot showing cluster assignments.

System Feature 4: Model Comparison Dashboard and Error Handling (5.4)

5.4.1 Description and Priority

Provides a side by side interface comparing metrics across multiple trained models and intercepts execution errors safely.

Priority: Medium.

Visualization can be easily done so the risk is low, but for error handling the risk involved is high because we may encounter various types of errors and we should be able to deal with all of them.

5.4.2 Stimulus/Response Sequences

- **User Action:** Selects multiple trained model runs to compare.
- **System Response:** Displays a consolidated side by side metric comparison dashboard.
- **User Action:** Submits an invalid dataset or configuration.
- **System Response:** Software throws an error.

5.4.3 Functional Requirements

REQ-12: The system should provide a comparison dashboard enabling users to view evaluation metrics of two or more trained models side by side.

REQ-13: The backend should intercept invalid execution states or invalid data formats and return error message to the UI.

Other Nonfunctional Requirements

6.1 Performance Requirements

The following performance requirements define the expected execution time and resource constraints of the Web-Based ML Model Evaluator.

- **PR-1: Pipeline Response Time**

The system shall complete dataset ingestion, feature scaling, model fitting, and metric generation within 30 seconds per run for datasets smaller than 10 MB.

- **PR-2: File Upload Constraint**

The system shall restrict the maximum size of uploaded dataset files to prevent excessive server load and performance degradation.

6.2 Safety Requirements

The following safety requirements define safeguards to prevent invalid or potentially harmful system operations during dataset processing and model evaluation.

- **SR-1: Safe Data Processing**

The system shall validate uploaded datasets and input configurations before processing to prevent invalid data from causing failures during preprocessing or model execution.

- **SR-2: Safe Model Execution**

The system shall prevent model execution when the selected task, features, target, or required parameters are invalid or incomplete.

- **SR-3: Error Handling**

The system shall handle invalid execution states without causing system failure and shall provide structured, human-readable error messages to the user interface.

6.3 Security Requirements

The following security requirements define measures for protecting user-provided datasets and preventing unauthorized or unsafe inputs from affecting the system.

- **SEC-1: Dataset Protection**

The system shall protect uploaded datasets from unauthorized access or disclosure during processing.

- **SEC-2: Input Validation**

The system shall validate uploaded files and user-provided inputs before processing to prevent malicious or invalid input from being executed by the system.

- **SEC-3: Secure Data Handling**

The system shall ensure that uploaded datasets and generated evaluation results are handled securely and shall not be exposed to unauthorized users.

- **SEC-4: Backend Security**

The backend shall reject requests containing invalid or unsupported inputs and shall return appropriate error messages without exposing internal system information.

6.4 Software Quality Attributes

The following quality attributes define the expected overall quality and usability of the Web-Based ML Model Evaluator.

- **QA-1: Usability**

The system shall provide a clear and intuitive workflow that guides users through the process of **dataset upload -> column selection -> model training -> result display**.

- **QA-2: Reliability**

The system shall handle invalid datasets, configurations, and execution states without causing system failure and shall provide appropriate feedback to the user.

- **QA-3: Portability**

The web application shall operate on modern web browsers, including Google Chrome, Mozilla Firefox, and Microsoft Edge, without requiring additional browser plugins.

- **QA-4: Maintainability**

The system shall be organized into separate functional components for data ingestion and preprocessing, ML task configuration, model execution and evaluation, and model comparison, allowing individual components to be modified or extended without significantly affecting the others.

- **QA-5: Robustness**

The system shall validate user inputs and dataset conditions before performing preprocessing or model execution, and shall gracefully handle invalid inputs and execution errors.

- **QA-6: Testability**

The system shall provide clearly defined inputs and outputs for its major functional components so that dataset processing, model training, evaluation, and result generation can be independently tested.

6.5 Business Rules

The following business rules define the conditions under which the Web-Based ML Model Evaluator performs its major functions.

- **BR-1: Dataset Requirement**

The user shall upload a valid dataset before proceeding with dataset preview, preprocessing, or model training.

- **BR-2: Supervised Learning Configuration**

For Classification and Regression, the user shall select one target variable and one or more feature variables before model training.

- **BR-3: Clustering Configuration**

For Clustering, the user shall specify the number of clusters before K-Means model execution.

- **BR-4: Train/Test Split**

The **train/test split ratio** shall be configurable only for supervised learning tasks such as Classification and Regression.

- **BR-5: Task-Specific Algorithms**

The system shall execute algorithms according to the selected ML problem type:

1. **Classification:** Logistic Regression, Decision Tree Classifier, Random Forest Classifier, and KNN.
2. **Regression:** Linear Regression, Decision Tree Regressor, and Random Forest Regressor.
3. **Clustering:** K-Means Clustering.

- **BR-6: Task-Specific Evaluation**

The system shall display evaluation metrics appropriate to the selected ML task:

1. **Classification:** Accuracy, Precision, Recall, F1-Score, and confusion matrix.
2. **Regression:** MAE, MSE, RMSE, and R^2.
3. **Clustering:** Silhouette Score, Davies-Bouldin Index, and PCA-based visualization.

- **BR-7: Model Comparison**

Model comparison shall be performed only for trained models with available evaluation results, and the system shall present the selected models' evaluation metrics side-by-side.

Other Requirements

7.1 System Maintenance & Scalability

- Modular Pipeline Architecture: The core backend must decouple data processing routines using pandas, algorithm fitting pipelines like scikit-learn, and API routes to ensure independent maintenance and allow seamless integration of new ML models in future releases.
- Deprecations & Libraries: All third party Python package dependencies such as scikit-learn, pandas, and numpy) must be locked to specific stable version releases to prevent build breaks from breaking API changes.

7.2 Error Handling & Fault Tolerance

- The execution engine must capture and wrap dataset exceptions (e.g. non numeric data in regression targets, zero-variance columns, or convergence timeouts) without terminating the server process.
- User facing diagnostics: Server side computational errors must be caught by custom backend middleware and returned as structured, human readable error messages to the UI.

7.3 Data Privacy & Memory Management

- Uploaded tabular datasets and generated visualization artifacts must be processed in memory or stored in temporary session buffers.
- Uploaded CSV files and intermediate model objects must be automatically purged from host memory upon session termination to conserve server memory resources.

Appendix A: Glossary

- **SRS:** Software Requirements Specification
- **ML:** Machine Learning
- **CSV:** Comma Separated Values
- **Supervised Learning:** Machine learning tasks (Classification and Regression) where models are trained using labeled target data.
- **Unsupervised Learning:** Machine learning tasks (Clustering) that discover hidden patterns or groupings in unlabeled datasets.
- **PCA:** Principal Component Analysis, a dimensionality reduction technique.
- **MAE:** Mean Absolute Error
- **MSE / RMSE:** Mean Squared Error/Root Mean Squared Error
- **R^2 Score:** Coefficient of Determination
- **Silhouette Score:** A clustering metric measuring how similar an object is to its own cluster compared to other clusters (ranging from -1 to +1).
- **Davies-Bouldin Index:** A clustering evaluation index where lower values signify better separation and tighter cluster groupings.

Appendix B: Field Layouts

Table 1: Dataset Configuration & Preprocessing Inputs

Field	Length	Data Type	Description	Mandatory?
Dataset File	Max 10 MB	CSV file	The user selects CSV dataset file	Yes
ML Task Type	20	Dropdown	Target task: Classification, Regression or Clustering	Yes
Target Variable	60	String/ Dropdown	Selected column name to predict (Supervised tasks only)	Yes (Supervised) / No (Clustering)
Train/Test Split Ratio	4	Float	Percentage of data reserved for testing	Yes (Supervised)

B.2 Output Report Field Layouts

Table 2: Model Performance Evaluation Metrics Report

Metric Field Name	Data Type	Source/Task	Description
Model Name	String	All	Name of the evaluated algorithm

Accuracy Score	Float (0 to 1)	Classification	Ratio of correct predictions over total predictions
Precision Score	Float (0 to 1)	Classification	Ratio of true positives over total positive predictions
Recall Score	Float (0 to 1)	Classification	Ratio of true positives over total actual positives
F1-Score	Float (0 to 1)	Classification	Harmonic mean of precision and recall
Confusion Matrix Data	2D Array/ JSON	Classification	Counts of true vs. predicted class values
Mean Absolute Error (MAE)	Float	Regression	Average absolute difference between predicted and actual values
Mean Squared Error (MSE)	Float	Regression	Average squared difference between predicted and actual values
Root MSE (RMSE)	Float	Regression	Square root of the Mean Squared Error
R2 Score	Float	Regression	Coefficient of determination

Silhouette Score	Float	Clustering	Measure of cluster cohesion and separation
Davies-Bouldin Index	Float	Clustering	Ratio of within-cluster scatter to between-cluster separation

Appendix C: Requirement Traceability Matrix

Sl. No	Requirement ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	System Test Case ID
1.	FR-1: Dataset Upload	User uploads dataset					
2.	FR-2: Dataset preview	5 record preview of uploaded dataset					
3.	FR-3: Preprocessing and Cleaning	Preprocessig and Cleaning of data, removing missing values if any					
4.	FR-4: ML Task Routing	select ML problem type					
5.	FR-8: Task specific Algorithm Execution	Execution of algorithms					
6.	FR-12: Model Comparison Visualizations	View comparison metrics and charts					

Software Architecture and Design Specification

Project: Web Based ML Model Evaluator

Version: 1.0

Authors: Sanyam Devan Hiran, Shreya Patil, Satvik Phadke, Shreya Ajith (Team 13)

Date: 05-10-2026

Status: Draft

1. Introduction

1.1 Purpose

This Software Architecture and Design Specification describes the architecture and design of the **Web Based ML Model Evaluator**. It explains the system components, data flow, interfaces, and security considerations involved in uploading datasets, configuring ML tasks, training models, and evaluating results.

1.2 Scope

The system provides a web-based pipeline for **CSV upload, data preprocessing, ML task configuration, model execution, and result visualization**. It supports Classification, Regression, and Clustering using algorithms such as Logistic Regression, Decision Tree, Random Forest, KNN, and K-Means. The system also provides relevant performance metrics and visualizations.

1.3 Audience

This document is intended for **developers, QA and test engineers, security auditors, instructors, students, and maintenance teams** involved in developing, testing, evaluating, or maintaining the system.

1.4 Definitions

- **ML:** Machine Learning
- **CSV:** Comma-Separated Values
- **API:** Application Programming Interface
- **JSON:** JavaScript Object Notation

- **PCA:** Principal Component Analysis
- **STRIDE:** Security threat-modeling framework
- **TSX:** TypeScript XML used for the frontend
- **SADS:** Software Architecture and Design Specification
- **SRS:** Software Requirements Specification
- **RTM:** Requirements Traceability Matrix

2. Document Overview

2.1 How to use this document

This Software Architecture and Design Specification serves as the structural and design document for this project. It outlines the architectural components (high level design) required for the implementation, verification, and maintenance of the application, including:

- Architectural and Analysis Models: High level component design, sequence of execution, and ownership assignments.
- Design and UML Diagrams: Step by step UML sequence diagram detailing data payload and system interactions across the frontend and backend components.
- Threat Models and Security Architecture: Uses the STRIDE framework, detailing about mitigating Denial of Service (DoS), data leakage, and parameter validation.
- API Specifications and Interfaces: Comprehensive endpoint definitions, request/response formats, status codes, and centralized error handling contracts.

Documents to be viewed (role based):

- Developers: Section 3 (Architecture) and Section 4 (Design) to understand the structure of the project and build based on the requirements.
- QA & Test Engineers: Utilize Section 3.8 (Traceability) and Section 4.3 and 4.4 (API & Error Handling) to align test scenarios with system components.
- Security Auditors & Evaluators: Review Section 3.9 (Security Architecture) and Section 3.1 (Constraints) regarding session memory management and input size enforcement.

2.2 Related Documents

This specification document is related to the following project documents:

- Software Requirements Specification (SRS): Defines the complete set of functional requirements (FR1 through FR13), non functional constraints (30 second response latency, 10 MB file size limit), and business rules governing the Web Based ML Model Evaluator.
- Software Test Plan (STP): Details the testing strategy, test coverage across all four core modules, execution schedules, entry/exit criteria, and security/performance verification procedures.
- Requirements Traceability Matrix (RTM): Maps every functional requirement (FR) and non functional requirement (NFR) to its corresponding architectural module, design component, code file reference, and test case ID to ensure complete lifecycle coverage.

3. Architecture

3.1 Goals & Constraints Goals: Provide an easy-to-use pipeline executing ingestion, scaling, fitting, and metrics generation within a 30-second response latency per run. Constraints: The system strictly restricts file uploads to a maximum of 10 MB to prevent server slowdowns. No long-term external database persistence is used; all execution is handled via session-based in-memory processing.

3.2 Stakeholders & Concerns Customers (Students/Teachers): Require a guided, simple interface without the overhead of setting up local code. Developers (Team 13): Require a highly decoupled architecture divided exactly into four independent modules to allow Shreya A., Satvik P., Shreya P., and Sanyam H. to work concurrently without merge conflicts.

3.3 Component Architecture & Data Flow

Pipeline Stage	Module Name	Primary Owner	Core Technology	Input	Output / Data Contract
Stage 1	Data Ingestion & Preprocessing	Shreya Ajith	Rust, Polars	Raw CSV File (< 10 MB)	Standardized Cleaned Data JSON

Stage 2	ML Task Routing & Configuration	Satvik Phadke	TSX Frontend, Rust APIs	Cleaned Data JSON, User UI Input	Task Configuration Struct
Stage 3	Algorithm Execution & Metrics	Shreya Patil	Rust, Linfa	Cleaned Data JSON + Config Struct	Raw Results JSON Object
Stage 4	Visualization, Comparison & Errors	Sanyam Deven Hiran	TSX, Recharts/Plo		

3.4 Component Descriptions

To ensure absolute independence among the four developers, the system is architected into four decoupled pipeline modules spanning the TSX frontend and Rust backend, communicating strictly through standardized JSON data contracts:

- Data Ingestion and Preprocessing Module: Handles CSV upload, strict 10MB size validation, and file formatting. Analyzes the dataset via Rust to return row/column counts, data types, and a 5-row preview. Manages missing value cleaning using Polars. Abstraction: Receives a raw file and outputs a standardized "Cleaned Data Object".
- ML Task Routing & Configuration Module: Manages user selections in the TSX UI for the problem type (Classification, Regression, Clustering). Handles target/feature column selections, clustering parameters, and train/test split configurations. Abstraction: Consumes the "Cleaned Data Object" metadata and outputs a "Task Configuration Struct" to the Rust API.
- Algorithm Execution & Metric Computation Module: Executes the mathematical model fitting strictly in Rust (Logistic Regression, Decision Tree, Random Forest, KNN, K-Means). Computes raw numerical metrics (Accuracy, F1-Score, MAE, R2,

Silhouette Score). Abstraction: Consumes the "Cleaned Data Object" and "Task Configuration Struct" to output a "Raw Results Object".

- Visualization, Comparison & Error Handling Module: Transforms raw metrics into a TSX Model Comparison Dashboard. Renders Confusion Matrices and 2D PCA scatter plots using frontend charting libraries. Intercepts invalid execution states across all Rust backend routes and returns structured, human-readable error messages to the UI.

3.5 Chosen Architecture Pattern and Rationale

A Sequential Pipeline / Micro-Core Architecture was chosen. A monolithic design would cause severe development bottlenecks for a 4-person team. By enforcing strict data contracts between the four distinct layers, each team member can develop, unit test, and mock their TSX UI components and Rust backend handlers concurrently.

3.6 Technology Stack & Data Stores Backend: Rust, Axum or Actix-Web (API framework), Polars (for high-performance data manipulation replacing pandas), Linfa or SmartCore (for ML algorithms replacing scikit-learn). Storage: Transient session-based memory only. Host memory buffers are purged of uploaded CSVs and model artifacts upon session termination. Frontend: TSX (React/TypeScript/Next.js) supporting modern browsers (Chrome, Firefox, and Edge). Plotly.js or Recharts for rendering PCA scatter plots and matrices.

3.7 Risks & Mitigations Risk: Processing delays exceeding the 30-second pipeline response latency. Mitigation: Utilizing Rust's zero-cost abstractions and Polars' multi-threaded execution guarantees processing well under the time limit. Perform early performance testing with datasets nearing the 10 MB limit. Risk: Unstructured or corrupted CSV files breaking the pipeline. Mitigation: The Rust Error Handling middleware will safely catch panics, intercept invalid states, and return structured error messages to the TSX frontend.

3.8 Traceability to Requirements R1 (FR-1, FR-2, FR-3) -> Data Ingestion & Preprocessing Module R2 (FR-4, FR-5, FR-6, FR-7) -> ML Task Routing & Configuration Module R3 (FR-8, FR-9, FR-10) -> Algorithm Execution Module R4 (FR-11, FR-12, FR-2 Error Handling) -> Visualization, Comparison & Error Handling Module

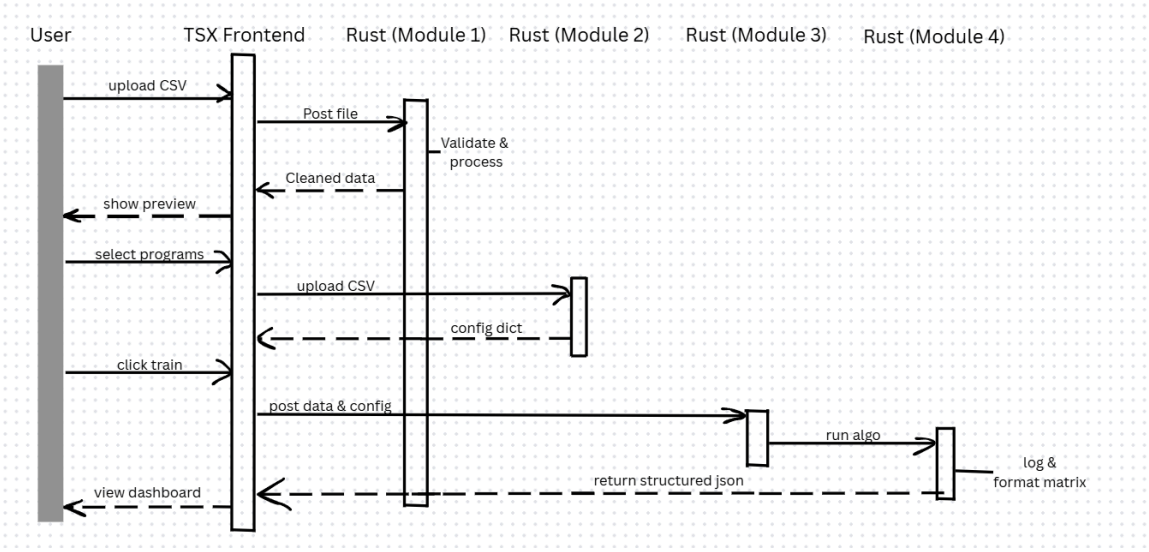
3.9 Security Architecture Threat Modeling (STRIDE): DoS -> rate limiting and enforcing strict file size boundaries with a hard 10 MB restriction. Data Leakage / Info disclosure -> purging host memory of uploaded CSVs upon session termination. Elevation of Privilege / Injection -> Validating input parameters strictly via Rust's strong type system and ensuring error messages do not expose sensitive host server details.

4. Design

4.1 Design Overview

The TSX frontend application and Rust backend are designed as a step-by-step wizard. The strict four-module pipeline prevents the execution module from firing before ingestion and configuration are fully validated, ensuring predictable data flows.

4.2 UML Sequence Diagram (Execution Flow)



4.3 API Design

Endpoint: /api/upload

Method: POST

Request: multipart/form-data {file: .csv}

Response: {status: string, row_count: u32, col_names: Vec, missing_vals: HashMap, preview: Vec}

Errors: 400 Invalid File Type, 413 Payload Too Large (>10MB).

Endpoint: /api/execute_model

Method: POST

Request: {problem_type: string, target_col: string, features: Vec, algo: string, test_split: f32}

Response: {metrics: HashMap, plot_data: Vec}

Errors: 422 Unprocessable Entity.

4.4 Error Handling, Logging & Monitoring A centralized Rust middleware interceptor catches invalid file formats, unsupported algorithm parameters, and missing data. It maps Rust Result/Option types into human-readable JSON error messages without exposing host server details to the TSX client.

4.5 UX Design The TSX UI follows a guided, clear process: upload -> select columns -> train model -> display results. The frontend is strictly typed with TypeScript and runs smoothly on Google Chrome, Firefox, and Microsoft Edge without requiring extra browser plugins.

4.6 Open Issues & Next Steps Completion of unit testing across the 4 separated pipelines, including Cargo tests for Rust and Jest/React Testing Library for TSX. Integration testing for the API interaction between the frontend and the Rust backend. Filling out the Requirements Traceability Matrix (RTM) completely during testing.

5. Appendices

5.1 Glossary

Term	Meaning
ATM	Automated Teller Machine
PIN	Personal Identification Number
PCI-DSS	Payment Card Industry Data Security Standard
TLS	Transport Layer Security
ADR	Architecture Decision Record

5.2 References

Reference	Full Title
IEEE 42010	ISO/IEC/IEEE 42010: Systems and software engineering, Architecture description
OWASP	Open Worldwide Application Security Project (Top Ten and security guidelines)
NIST SP 800-160	Systems Security Engineering: Considerations for a Multidisciplinary Approach

5.3 Tools

Tool	Purpose
PlantUML	Text-based UML diagram generation (e.g. sequence diagram in Section 4.2)
draw.io	Architecture and pipeline diagrams
Swagger	API specification and documentation (Section 4.3)

Software Test Plan (STP) - Web Based ML Model Evaluator

Project: Web based ML Model Evaluator
Version: 1.0
Authors: Shreya Patil, Satvik Phadke, Shreya Ajith, Sanyam Deven Hiran (group 13)
Date: 17-09-2025
Status: Sample / Draft

1. Introduction

Purpose: This document defines the test strategy, scope, schedule, and responsibilities for validating the Web Based ML Model Evaluator v1.0, covering the objectives, scope, strategy, resources, schedule, and responsibilities for verifying data ingestion, data preprocessing, machine learning task routing, model execution, performance evaluation, visualization, side-by-side comparison, and non-functional requirements.

Scope: Testing covers end-to-end browser features including CSV uploads (<10 MB), preview/summary, missing value handling, task routing (Classification, Regression, Clustering), parameter configuration, algorithm execution, visualizations (Confusion Matrix, 2D PCA plots), comparison dashboard, error handling, and security/performance constraints. Excludes host hardware and internal logic of third-party libraries (e.g., scikit-learn, pandas).

References:
- Software Requirements Specification (SRS) for Web Based ML Model Evaluator v1.0 (Group 13)
- Functional Requirements Document - Web Based ML Model Evaluator (Team 13)
- IEEE Recommended Practice for Software Requirements Specifications

Definitions:
- **ML:** Machine Learning
- **SRS:** Software Requirements Specification
- **CSV:** Comma Separated Values
- **PCA:** Principal Component Analysis
- **KNN:** K-Nearest Neighbors
- **MAE / MSE / RMSE:** Mean Absolute Error / Mean Squared Error / Root Mean Squared Error
- **RTM:** Requirements Traceability Matrix
- **UAT:** User Acceptance Testing

2. Test Items

1. Data ingestion and preprocessing module
- Includes file upload, file preview and basic missing value handling.

2. ML task routing and configuration module
- Includes selecting the problem type (classification, regression and clustering), target selection, cluster count inputs, train/test splits inputs (if the problem type requires it).

3. Algorithm execution and metric computation module
- Execute the problem type and calculate the metrics.

4. Model comparison and visualization module
- Side by side comparison of multiple trained models, display of confusion matrix, plots and calculated metrics.

5. Error handling
- Error handling will be a separate component in the project.
- Checking invalid file formats, ensuring size limits, etc and structure and send the error message to the UI.

3. Features to be Tested

- FR-2: 5 row data preview of uploaded csv file
- FR-3: Preprocessing and missing value handling
- FR-4: ML Task Selection such as classification, regression, clustering
- FR-5: Feature and target selection for supervised learning tasks
- FR-6: Feature selection and cluster count configuration for clustering specific problem
- FR-7: Train/test data partitioning from user
- FR-8 / BR-5: Task specific algorithm execution

  Eg:

  Classification: Logistic regression, decision tree, KNN, etc

  Regression: Linear regression, decision tree regressor, etc

  Clustering: K-Means clustering

- FR-9 / BR-6: Classification metrics & visualizations (accuracy, precision, recall, F1 score, confusion matrix)
- FR-10 / BR-6: Regression metrics (MAE, MSE, RMSE, R^2 score)
- FR-11 / BR-6: Clustering metrics & visualizations (silhouette score, davies-bouldin Index, PCA scatter plot)
- FR-12 / BR-7: Model comparison dashboard having side by side metrics comparison
- FR-2 / SR-3 / SEC4: Error handling such as unsupported file types, unsupported algorithm specific user inputs and exceeding file size limit.

4. Features Not to be Tested

- Internal logic of underlying standard python libraries (scikit, numpy etc.)
- Data formats other than csv (non-csv upload rejection is tested under input validation)
- Physical web server hardware maintenance, host OS security, and cloud data center network infrastructure.

- Long term external database persistence (application uses session based in memory processing)

5. Test Approach / Strategy

Levels:
- **Unit Testing:** Validating individual backend functions (CSV parsing, pre-processing functions, metric calculation scripts, route handlers)
- **Integration Testing:** Testing API interaction between UI frontend and FastAPI/Python backend pipeline.
- **System Testing:** End-to-end evaluation from CSV upload to preprocessing to model configuration to execution to visualization and side-by-side comparison
- **Acceptance Testing (UAT):** Verifying the workflow simplicity and evaluation of metric correctness against business rules with domain user scenarios.

Types:
- **Functional Testing:** Verifying all REQ-1 through REQ-13 and FR-1 through FR-12 capabilities
-**Boundary & Negative Testing:** Uploading files exceeding 10 MB, invalid CSV structures, missing columns, selecting same target as feature, non-numeric regression targets.
- **Performance Testing:** Measuring execution times to ensure <30 second latency for <10 MB datasets.
- **Usability & Compatibility Testing:** Ensuring guided 4-step workflow rendering across Chrome, Firefox, and Edge .

- **Security & Memory Testing:** Verifying dataset protection, input sanitization, and session buffer memory purge.

Entry Criteria:

- Stable web application build deployed in test environment

- Test CSV datasets (Classification, Regression, Clustering scenarios) prepared.

- SRS and Functional Specifications baselined.

Exit Criteria: 100% of planned test cases executed, 0 critical defects open, all acceptance criteria satisfied.

5.1 Security Validation

- Validate input file type checking (strictly enforcing .csv files).
- Enforce file size boundaries with a hard 10 MB restriction to prevent denial-of-service (DoS) or resource exhaustion.
- Validate input parameters to prevent injection attacks or invalid configuration payloads.
- Verify session buffer cleanup by purging host memory of uploaded CSVs and model artifacts upon session termination.
- Validate error messages to ensure backend traces do not expose sensitive host server details.

6. Test Environment

Hardware: Test Workstations (Intel i5/i7, 16GB RAM)
OS: Windows 11 / Linux Ubuntu.
Backend Environment: Python 3, FastAPI framework, pandas, scikit-learn, numpy
Tools:
- API Testing: Postman
- Performance Testing: Apache JMeter / Locust
- Defect Tracking & Management: Jira

Test Data: Standard benchmark CSV datasets under 10 MB, corrupted/malformed CSV files, zero-variance feature datasets, non-numeric regression datasets.

7. Test Schedule

Milestones:
- Test case design: 05-Oct-2025
- Environment setup: 07-Oct-2025
- Test execution start: 08-Oct-2025
- Test execution end: 20-Oct-2025
- UAT: 22-Sep-2025 to 25-Oct-2025

8. Test Deliverables

- Software Test Plan (this document)
- Test Cases Specification Document (Manual & Automated Scripts)

- Requirements Traceability Matrix (RTM)
- Test Data Sets (Valid & Synthetic Invalid CSVs)
- Execution Logs & Performance Metrics Reports
- Defect Reports (Jira export)
- Final Test Summary Report

9. Roles and Responsibilities

Role	Name	Responsibility
QA Lead	Sanyam Deven Hiran	Prepare plan, coordinate execution
Test Engineer	Shreya Patil, Shreya Ajith, Satwik Phadka, Sanyam Deven Hiran	Design & execute test cases, log defects
Developer	Shreya Patil, Shreya Ajith, Satwik Phadka, Sanyam Deven Hiran	Support defect fixes and triage
Product Owner	Sudeepa Roy Dey	Approve test results, sign-off readiness

10. Risks and Mitigation

Risk	**Mitigation**
Processing delays exceeding the 30-second pipeline response latency limit for model training (NFR-1).	Perform early performance testing with datasets nearing the 10 MB limit and optimize backend model configurations.
File upload failures or server slowdowns due to excessively large datasets.	Enforce and test strict maximum file size restrictions (NFR-4) before processing begins.

Unstructured or corrupted CSV files breaking the preview (FR-2) or preprocessing (FR-3) modules.	Ensure robust error handling is tested to verify it intercepts invalid states and returns structured, human-readable error messages to the UI (FR-2).
Browser compatibility issues leading to UI rendering failures.	Execute cross-browser testing on Google Chrome, Firefox, and Microsoft Edge early in the test cycle (NFR-3).

11. Assumptions & Dependencies

- **Assumptions:**
  - Users will upload datasets in valid CSV formats suitable for analysis.
  - The user interface follows the defined clear process of upload, selecting columns, training the model, and displaying results (NFR-2).
  - The selected target variable and features provided by the user will be appropriate for the chosen ML problem type.
- **Dependencies:**
  - Availability of modern web browsers (Google Chrome, Firefox, Microsoft Edge) for UI validation without requiring extra plugins (NFR-3).
  - Availability of sample test datasets (under 10 MB) suitable for Classification, Regression, and Clustering tasks.
  - Stability of backend machine learning algorithms (e.g., Logistic Regression, Random Forest, K-Means, etc.) required for task-specific execution (FR-8).

12. Suspension & Resumption Criteria

- **Suspend testing if:**
  - The web application environment is unavailable for >4 hours.
  - Core functionality defects block essential pathways, such as the inability to upload a dataset (FR-1) or select an ML task (FR-4).
  - The system consistently crashes instead of returning structured error messages during invalid execution states (FR-2).
- **Resume testing if:**
  - Critical blocking defects (like file upload failures or dataset preview crashes) are resolved by the development team.
  - The test environment is stabilized and accessible.

13. Test Case Management & Traceability

Requirement Traceability Matrix (RTM):

Sl. No.	Requirement ID	Brief Description of Requirement	Architecture Reference	Design Reference	Code File Reference	Test Case ID	Pass/ Fail
1	FR-1	User should be able to upload their dataset file here.	AM-1: Dataset Ingestion and Preprocessing	SF-1: Data Ingestion and Preprocessing	TBd	T1	TBD
2	FR-2	System should analyze the uploaded CSV and present a 5 record dataset preview and show total rows, column names, data types, missing value	AM-1: Dataset Ingestion and Preprocessing	SF-1: Data Ingestion and Preprocessing	TBD	T2	TBD
3	FR-3	System should preprocess and clean the dataset	AM-1: Dataset Ingestion and Preprocessing	SF-1: Data Ingestion and Preprocessing	TBD	T3	TBD
4	FR-4	System should allow the user to select an ML problem type such as classification or regression	AM-2: ML Task Configuration	SF-2: ML Task Routing and Configuration	TBD	T4	TBD
5	FR-5	For classification and regression tasks, the system should allow the user to select one column as the target variable and one or more columns as features.	AM-2: ML Task Configuration	SF-2: ML Task Routing and Configuration	TBD	T5	TBD

6	FR-6	For clustering, the system should allow the user to input the number of clusters.	AM-2: ML Task Configuration	SF-2: ML Task Routing and Configuration	TBD	T6	TBD
7	FR-7	For supervised tasks, the system shall provide an option to adjust the train/test split ratio.	AM-2: ML Task Configuration	SF-2: ML Task Routing and Configuration	TBD	T7	TBD
8	FR-8	System should execute model fitting based on user selection-classification, regression and clustering algorithms.	AM-3: Model Training and Evaluation	SF-3: Algorithm Execution and Metric Computation	TBD	T8	TBD
9	FR-9	System should compute and display accuracy, precision, recall, F1 score, and a confusion matrix for classification.	AM-3: Model Training and Evaluation	SF-3: Algorithm Execution and Metric Computation	TBD	T9	TBD
10	FR-10	System should compute and display MAE, MSE, RMSE, and R^2 where applicable.	AM-3: Model Training and Evaluation	SF-3: Algorithm Execution and Metric Computation	TBD	T10	TBD
11	FR-11	System should compute Silhouette Score and Davies-Bouldin Index and display a 2D PCA scatter plot showing cluster assignments.	AM-3: Model Training and Evaluation	SF-3: Algorithm Execution and Metric Computation	TBD	T11	TBD
12	FR-12	System should allow users to compare evaluation metrics of two or more trained models side by side.	AM-4: Model Comparison	SF-4: Model Comparison Dashboard	TBD	T12	TBD
13	FR-13	Backend should intercept invalid execution states and return error	AM-1, AM-2, AM-3, AM-4	SF-4: Error Handling	TBD	T13	TBD

		messages to the UI.					
14	NFR-1	Ingestion, feature scaling, model fitting, and metric generation for datasets under 10 MB shall execute within 30 seconds per run.	Overall Architecture	Performance Requirements	TBD	T14	TBD
15	NFR-2	The web interface should guide the user through a clear process	AM-1, AM-2, AM-3, AM-4	UI / Usability Design	TBD	T15	TBD
16	NFR-3	The web application should run smoothly on Google Chrome, Firefox, and Microsoft Edge without requiring extra browser plugins.	Overall Architecture	External Interface / Browser Compatibility Design	TBD	T16	TBD
17	NFR-4	The system should restrict file uploads to a maximum size to prevent server slowdowns.	AM-1: Dataset Ingestion and Preprocessing	SF-1: Data Ingestion and Preprocessing	TBD	T17	TBD

14. Test Metrics & Reporting

Metrics:

- % Test Cases Executed vs. Planned
- % Pass / Fail Rate
- Defect Density (Defects per Functional Module)
- Defect Severity Breakdown (Critical, Major, Minor)
- Pipeline Execution Latency Timings (seconds per dataset)
- Requirement Coverage Percentage (Target: 100%)

Reports:

- Daily Test Execution Status Update
- Weekly Defect Tracking & Triage Summary
- Final Test Sign-Off Summary Report

15. Approvals

Role	Name	Signature / Date
QA Lead	Sanyam Deven Hiran	
Test Engineer	Shreya Patil, Shreya Ajith, Satwik Phadka, Sanyam Deven Hiran	
Developer	Shreya Patil, Shreya Ajith, Satwik Phadka, Sanyam Deven Hiran	
Product Owner	Sudeepa Roy Dey	